

7

Expressions and Assignment Statements

- 7.1 Introduction
- 7.2 Arithmetic Expressions
- 7.3 Overloaded Operators
- 7.4 Type Conversions
- 7.5 Relational and Boolean Expressions
- 7.6 Short-Circuit Evaluation
- 7.7 Assignment Statements
- 7.8 Mixed-Mode Assignment

Türkçe Anlatım — Terimler İngilizce

Konu Anlatımı + Soru Çözümleri

İçindekiler

1 Giriş (Introduction)	2
2 Arithmetic Expressions	3
2.1 Operator Evaluation Order	3
2.1.1 Precedence (Öncelik)	3
2.1.2 Associativity (Eşleştirme Yönü)	3
2.1.3 Parentheses (Parantezler)	4
2.2 Operand Evaluation Order	4
2.2.1 Referential Transparency	4
2.3 Özel Dil Özellikleri	5
3 Overloaded Operators	6
4 Type Conversions	7
4.1 Widening ve Narrowing	7
4.2 Coercion in Expressions	7
4.3 Explicit Type Conversion (Cast)	7
5 Relational and Boolean Expressions	8
5.1 Relational Expressions	8
5.2 Boolean Expressions	8
6 Short-Circuit Evaluation	9
7 Assignment Statements	10
7.1 Simple Assignments	10
7.2 Assignment Çeşitleri	10
7.3 Compound Assignment Operators	10
7.4 Unary Assignment Operators (++ , --)	10
7.5 Assignment as an Expression	11
7.6 Multiple Assignments (Perl, Ruby)	11
7.7 Assignment in Functional Programming	11
8 Mixed-Mode Assignment	12
9 Review Questions — Soru Çözümleri	14
10 Problem Set — Soru Çözümleri	21

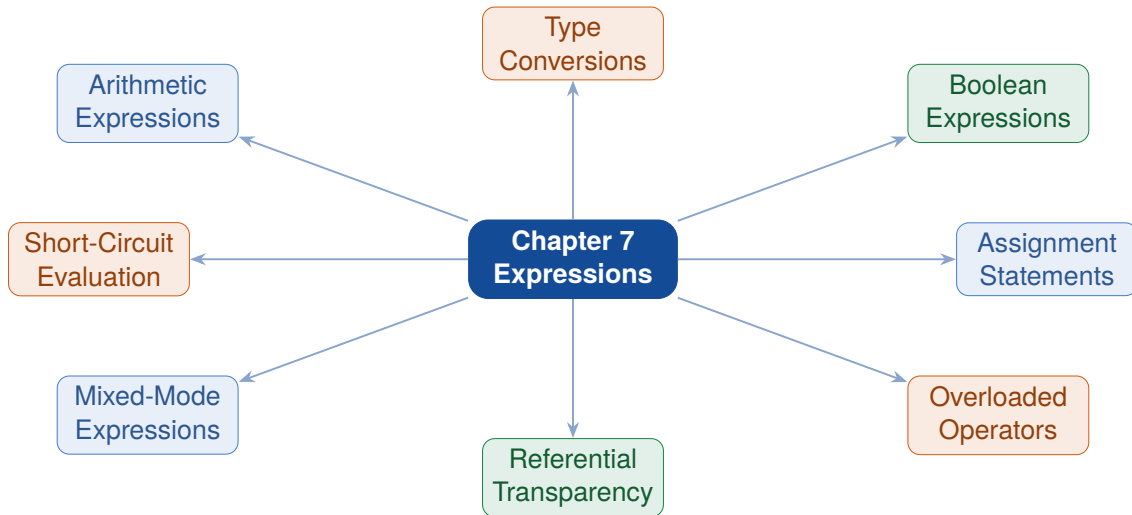
1 Giriş (Introduction)

Temel Kavram

Expression (ifade), bir programlama dilinde hesaplama belirtmenin temel yoludur. Bir expression; **operator**ler, **operand**lar, parantezler ve fonksiyon çağrılarından oluşur. **Assignment statement (atama ifadesi)** ise değişkenlerin değerlerini değiştirmek için kullanılan temel yapı taşıdır.

Expression değerlendirilirken anlaşılması gereken kritik konular:

- **Operator precedence** ve **associativity** kuralları
- **Operand evaluation order** ve **side effect**'lerin etkileri
- **Type mismatch**, **coercion** ve **short-circuit evaluation**



2 Arithmetic Expressions

2.1 Operator Evaluation Order

2.1.1 Precedence (Öncelik)

Tanım: Operator Precedence

Operator precedence, farklı öncelik düzeylerine sahip operatörlerin bir expression içinde hangi sırada değerlendirileceğini belirler. Matematik kökenli bu kurallar hemen her imperative dilde benzerdir: üs alma en yüksek, ardından çarp/böl, son olarak topla/çıkart.

Öncelik Düzeyi	Ruby	C-tabanlı Diller
En Yüksek	<code>**</code> unary <code>+</code> , <code>-</code>	postfix <code>++</code> , <code>--</code> prefix <code>++</code> , <code>--</code> , unary <code>+</code> , <code>-</code>
Orta	<code>*</code> , <code>/</code> , <code>%</code>	<code>*</code> , <code>/</code> , <code>%</code>
En Düşük	<code>+</code> , <code>-</code>	<code>+</code> , <code>-</code>

Örnek: Precedence Etkisi

```

1 // a=3, b=4, c=5 için:
2 a + b * c
3 // Soldan ğsaa (topla önce): (3+4)*5 = 35 -- YANLIŞ
4 // Precedence kuralına göre: 3+(4*5) = 23 -- DOĞRU

```

2.1.2 Associativity (Eşleştirme Yönü)

Tanım: Operator Associativity

Aynı öncelik düzeyindeki **iki bitişik operatör** bulunduğunda, hangisinin önce değerlendirildiği **associativity** kuralıyla belirlenir.

- **Sol associativity:** Soldaki operatör önce değerlendirilir. Örnek: $a - b + c \Rightarrow (a-b)+c$
- **Sağ associativity:** Sağdaki operatör önce değerlendirilir. Örnek (Fortran): $A**B**C \Rightarrow A**(B**C)$

Dil	Sol Associativity	Sağ Associativity
Ruby	<code>*</code> , <code>/</code> , <code>+</code> , <code>-</code>	<code>**</code>
C-tabanlı	<code>*</code> , <code>/</code> , <code>%</code> , binary <code>+/ -</code>	<code>++</code> , <code>--</code> , unary <code>+/ -</code>
APL	—	Tüm operatörler
Visual Basic	<code>^</code> dahil hepsi	—

APL İstisnası

APL'de **tek bir öncelik düzeyi** vardır ve tüm operatörler **sağdan sola** değerlendirilir. Bu nedenle $A * B + C$ ifadesi matematiksel sezginin aksine $A * (B+C)$ olarak hesaplanır.

2.1.3 Parentheses (Parantezler)

Parantezler, precedence ve associativity kurallarını geçersiz kılar. Parantezli kısım, bitişik parantezsiz kısımlara göre her zaman **daha yüksek öncelik** kazanır.

Örnek

```
1 (A + B) * C           // Toplama önce yapılır
2 (A + B) + (C + D)    // Overflow'u önlemek için yeniden gruplama
```

2.2 Operand Evaluation Order**Dikkat: Side Effect Problemi**

Eğer bir fonksiyon çağırısı **side effect** (yan etki) yaratıyorsa, operandların hangi sırada değerlendirileceği expression sonucunu etkiler. Çoğu dil bu sıralamayı tanımlamaz; bu da farklı implementasyonlarda farklı sonuçlara yol açar. **Java**, operand değerlendirmesinin soldan sağa yapılacağını garanti eder.

Kod	Soldan Sağa	Sağdan Sola
<pre>a = 10; b = a + fun(&a); (fun → a =20, döner 3)</pre>	<pre>1. a okunur: 10 2. fun çağırılır: a =20, 3 döner Sonuç: b = 10+3 = 13</pre>	<pre>1. fun çağırılır: a =20, 3 döner 2. a okunur: 20 Sonuç: b = 20+3 = 23</pre>

2.2.1 Referential Transparency**Tanım: Referential Transparency**

Bir programda aynı değere sahip iki expression birbirinin yerine **programın çalışmasını değiştirmeden** kullanılabiliriyorsa, program **referentially transparent** özelliğindedir. Pure functional dillerde değişken olmadığı için bu özellik doğal olarak sağlanır.

Örnek: Referential Transparency

```
1 // fun yan etki yaratmıyorsa: result1 == result2
2 result1 = (fun(a) + b) / (fun(a) - c);
3 temp    = fun(a);
```

```

4 result2 = (temp + b) / (temp - c);
5 // ğEer fun, b veya c'yi ğşdeitiriyorsa → result1 != result2
6 // => Referential transparency ŞBOZULMU

```

2.3 Özel Dil Özellikleri

Ruby — Operatörler Method Olarak

Ruby’de `+`, `-`, `*` gibi tüm aritmetik operatörler **method** olarak implement edilmiştir. Örneğin `a + b`, `a`’nın `+` methodunu `b` parametresiyle çağırarak eşdeğerdir. Bu sayede operatörler uygulama programları tarafından **override** edilebilir.

Lisp — Prefix Notation

Lisp’te tüm aritmetik işlemler subprogram olarak gerçekleştirilir ve **prefix** yazım zorunludur:

$$a + b * c \longrightarrow (+ a (* b c))$$

Conditional Expressions (Üçlü Operatör)

C-tabanlı dillerde **conditional expression** (üçlü/ternary operatör):

```
koşul ? deger_eger_dogru : deger_eger_yanlis
```

Örnek: `average = (count == 0) ? 0 : sum / count;`
 Perl, JavaScript ve Ruby’de de desteklenir.

3 Overloaded Operators

Tanım: Operator Overloading

Operator overloading, bir operatör sembolünün birden fazla anlam için kullanılmasıdır. Örneğin `+` operatörü çoğu dilde hem tam sayı hem de kayan nokta toplamı için kullanılır; Java'da ek olarak string birleştirme için de kullanılır.

Operatör	Kullanım 1	Kullanım 2
<code>+</code>	Integer toplama	Float toplama
<code>+</code> (Java)	Integer/Float toplama	String concatenation
<code>-</code>	Binary çıkarma	Unary negatif
<code>&</code> (C++)	Binary bitwise AND	Unary address-of
<code>*</code>	Çarpma	User-defined (matris vb.)

User-Defined Operator Overloading

C++, C# ve F# programcılarının kendi operatör tanımlarını yapmasına izin verir.

Matris İşlemleri Örneği (C++)

```

1 // + ve * matris için tanımlanmışsa:
2 A * B + C * D
3 // yazmak yeterlidir. Aksi hâlde:
4 MatrixAdd(MatrixMult(A, B), MatrixMult(C, D))
5 // yazmak gerekirdi.

```

Overloading'in Riskleri

- Hiçbir şey `+`'ı çarpma olarak tanımlamayı engellemez.
- Farklı modüller aynı operatörü farklı şekilde overload edebilir.
- Operatör anlamını anlamak için operand tiplerini ve tüm tanımları bulmak gerekir.
- Java, user-defined overloading'i **kaldırdı**; C# ise yeniden **ekledi**.

4 Type Conversions

4.1 Widening ve Narrowing

Dönüşüm Türü	Açıklama	Örnek (Java)
Widening	Tüm değerleri temsil edebilen büyük tipe dönüşüm. Genellikle güvenlidir .	<code>int → float</code>
Narrowing	Büyük tipten küçüğe; veri kaybı olabilir.	<code>double → float</code>
Coercion	Derleyici tarafından otomatik (implicit) dönüşüm.	<code>int a; float b = a;</code>
Cast	Programcı tarafından açıkça belirtilen dönüşüm.	<code>(int) 3.7 → 3</code>



4.2 Coercion in Expressions

Coercion ve Hata Tespiti

Mixed-mode expression'larda derleyici, uyumsuz tipleri otomatik olarak dönüştürür. Bu durum **hata tespitini** azaltır:

```

1 int a;
2 float b, c, d;
3 d = b * a; // 'a' yerine 'c' yazılması gerekirdi -- derleyici
             HATA VERMEZ
  
```

Bu nedenle **F#, Ada, ML** mixed-mode expression'lara *izin vermez*.

4.3 Explicit Type Conversion (Cast)

Cast Sözdizimi Karşılaştırması

```

1 // C / C++ / Java / C#:
2 (int) angle           // parantez + tip adı
3
4 // ML / F#:
5 float (sum)           // fonksiyon çağırışı şeklinde
  
```


5 Relational and Boolean Expressions

5.1 Relational Expressions

Tanım

Relational operator, iki operandın değerini karşılaştırır. Bir **relational expression** iki operand ve bir relational operatörden oluşur; sonucu **Boolean**'dır.

Karşılaştırma	C/Java	Fortran 95+	ML/F#	Ruby
Eşit	==	.EQ. / =	=	==
Eşit Değil	!=	.NE. / <>	<>	!=
Küçük	<	.LT. / <	<	<
Büyük	>	.GT. / >	>	>
Coercion'sız eşit	===	---	---	eql?

JavaScript / PHP: === vs ==

```
1 "7" == 7 // true -- string "7", ısayya coerce edilir
2 "7" === 7 // false -- coercion ıyaplmaz, ıfarkl tipler
```

5.2 Boolean Expressions

C99 öncesinde C'nin ayrı bir **Boolean tipi yoktu**: sıfır **false**, sıfır olmayan her değer **true** sayılıyordu.

Operasyon	C-tabanlı	Ada	Python	Ruby
AND	&&	and	and	&& / and
OR		or	or	/ or
NOT	!	not	not	! / not
Bitwise AND	&	---	&	&
Bitwise OR		---		

C-tabanlı dillerde tam öncelik sırası (en yüksekten en düşüğe):

Sıra	Operatörler
En Yüksek	postfix ++, --
2	unary +, unary -, prefix ++, --, !
3	*, /, %
4	binary +, binary -
5	<, >, <=, >=
6	==, !=
7	&& (AND)
En Düşük	(OR)

6 Short-Circuit Evaluation

Tanım

Short-circuit evaluation: Bir Boolean expression'ın tüm operand ve operatörleri değerlendirilmeden sonucun belirlenebilmesi durumu.

- `FALSE && X` → Her zaman `FALSE`; `X` değerlendirilmez.
- `TRUE || X` → Her zaman `TRUE`; `X` değerlendirilmez.

AND (&&) Short-Circuit	OR () Short-Circuit
1. operand FALSE ise: → 2. operand ATLANDI → Sonuç: FALSE	1. operand TRUE ise: → 2. operand ATLANDI → Sonuç: TRUE
1. operand TRUE ise: → 2. operand değerlendirilir	1. operand FALSE ise: → 2. operand değerlendirilir

Short-Circuit Olmadan Dizi Arama — Hata

```

1 // Java - short-circuit ııkullanılmasaydı:
2 index = 0;
3 while ((index < listlen) && (list[index] != key))
4     index = index + 1;
5 // index == listlen olduğunda: list[listlen] -> SUBSCRIPT
  HATASI!
6 // Short-circuit sayesinde: index >= listlen ise
7 // ikinci ııksm değerlendirilmez -> GÜVENLİ

```

Short-Circuit + Side Effect = Tehlikeli Kombinasyon

```

1 // Java:
2 (a > b) || ((b++) / 3)
3 // Eğer a > b ise: (b++) HIÇBİR ZAMAN ııııçaltrılmaz
4 // Program ııııdoruluu b'nin her seferinde ıartmasna
5 // ııbalysa: HATA

```

Dil	Short-Circuit Davranışı
Java, C#	<code>&&</code> ve <code> </code> short-circuit; <code>&</code> ve <code> </code> değil
Python	Tüm logical operatörler short-circuit
Ruby, Perl, ML, F#	Tüm logical operatörler short-circuit
Ada	<code>and/or</code> değil; <code>and then/or else</code> short-circuit

7 Assignment Statements

7.1 Simple Assignments

Hemen hemen tüm modern diller atama için `=` kullanır. ALGOL 60 ve Ada'nın tercih ettiği `:=` ise atama ile eşitlik karışmasını önler.

7.2 Assignment Çeşitleri

Tür	Sözdizimi	Açıklama
Simple	<code>a = b + c</code>	Temel atama
Conditional Target	<code>(\$flag?\$a:\$b) = 0</code>	Koşullu hedef (Perl)
Compound	<code>sum + = value</code>	<code>sum = sum + value</code> kısaltması
Unary	<code>count++, --x</code>	1 artır/azalt
As Expression	<code>while((ch = getchar()) != EOF)</code>	Atama değer üretir (C)
Multiple	<code>(\$a, \$b) = (\$b, \$a)</code>	Çoklu kaynak-hedef (Perl/Ruby)

7.3 Compound Assignment Operators

Compound Assignment

```

1 sum + = value; // sum = sum + value; ile şğedeer
2 a - = 3; // a = a - 3;
3 x * = 2; // x = x * 2;
4 n / = 4; // n = n / 4;
5 r % = 7; // r = r % 7;

```

ALGOL 68 tarafından tanıtılmış; C, Python, JavaScript, Ruby vb.'de desteklenir.

7.4 Unary Assignment Operators (++ , --)

Kullanım	Anlam	Eşdeğerliği
<code>sum = ++count</code>	Önce artır, sonra ata	<code>count = count+1; sum = count;</code>
<code>sum = count++</code>	Önce ata, sonra artır	<code>sum = count; count = count+1;</code>
<code>count++</code>	Sadece artır	<code>count = count + 1;</code>
<code>-count++</code>	Önce artır, sonra negat	<code>-(count++)</code>

7.5 Assignment as an Expression

Tehlike: = vs == Karışırması (C/C++)

```

1 // 1şYanl 1yazm: == yerine = 1kullanld
2 if (x = y) ... // Hata ğdeil! y x'e 1atanr, sonra test
   edilir
3 if (x == y) ... // ğDoru: Őeitlik testi
4
5 // Java ve C# bunu önler: if içinde sadece
6 // boolean expression kabul edilir

```

Güvenli Kullanım — while ile getchar

```

1 while ((ch = getchar()) != EOF) {
2     // ch'ye karakter okunur ve EOF ile ŐıŐı11karlatrlr
3 }
4 // Parantez zorunlu: çünkü = operatörü, !='den
5 // ŐDÜÜK önceliktedir.

```

7.6 Multiple Assignments (Perl, Ruby)

Çoklu Atama — Perl

```

1 ($first, $second, $third) = (20, 40, 60);
2 # ğDeer ğŐdeitirme -- geçici ğŐdeiken GEREKMEZ:
3 ($first, $second) = ($second, $first);

```

7.7 Assignment in Functional Programming

Fonksiyonel dillerde (ML, F#) atama; bir isme değeri **bağlamak (binding)** anlamındadır, değeri **değiştirmek** değil. ML'de `val`, F#'de `let` kullanılır; değeri hiçbir zaman değerişmez.

8 Mixed-Mode Assignment

Tanım

Atama ifadesinin sol tarafındaki değişkenin tipi ile sağ taraftaki expression'ın tipi farklı ise **mixed-mode assignment** gerçekleşmiştir.

Dil	Kural	Güvenlik
C, C++, Perl	Çok geniş coercion; çoğu tip karışımına izin verir.	Düşük
Java, C#	Sadece widening coercion'a izin verir.	Yüksek
Ada, ML, F#	Hiç mixed-mode yoktur.	En Yüksek
Python, Ruby	Tipler nesnelere aittir; mixed-mode kavramı yoktur.	—

Java Örneği

```

1 int    i = 5;
2 float f = i;      // İİİZNL -- widening (int -> float)
3
4 double d = 3.14;
5 int    n = d;      // HATA -- narrowing, cast gerekli
6 int    m = (int) d; // İİİZNL -- 1açk cast ile

```

Bölüm Özeti

Chapter 7 — Temel Kavramlar Özeti

- **Operator Precedence:** Farklı öncelikli operatörlerin değerlendirme sırası. Matematik kökenli; hemen her dilde benzerdir.
- **Operator Associativity:** Aynı öncelikteki iki bitişik operatör için hangi tarafın önce değerlendirileceği. Çoğu dilde soldan sağa.
- **Side Effect:** Bir fonksiyonun parametresini veya global değişkeni değiştirmesi. Operand sırasıyla etkileşince tutarsız sonuçlara yol açar.
- **Referential Transparency:** Aynı değerli iki expression birbirinin yerini alabilirse sağlanır. Functional dillerde doğaldır.
- **Operator Overloading:** Bir sembolün birden fazla anlama gelmesi. Kullanıcı tanımlı overloading C++, C#, F#’de desteklenir.
- **Widening / Narrowing:** Widening güvenlidir; narrowing veri kaybı riski taşır. Java/C# sadece widening mixed-mode’a izin verir.
- **Coercion:** Derleyici tarafından otomatik tip dönüşümü; hata tespitini azaltır.
- **Short-Circuit Evaluation:** İlk operanddan sonuç belirlenirse ikinci operand değerlendirilmez.
- **Compound Assignment:** `sum + = value` gibi kısaltma operatörleri.
- **Multiple Assignment:** Perl ve Ruby’de geçici değişken kullanmadan değer değişimi.

9 Review Questions — Soru Çözümleri

S1. Define operator precedence and operator associativity.

Cevap

Operator precedence (operatör önceliği): Farklı öncelik seviyelerindeki operatörlerin bir expression içinde hangi sırada değerlendirileceğini belirleyen kuraldır. Örneğin $*$ operatörü $+$ 'dan yüksek öncelikte olduğu için $a + b * c$ ifadesinde önce çarpma yapılır: $a + (b * c)$.

Operator associativity (operatör eşleştirme yönü): Aynı öncelik düzeyindeki iki bitişik operatörün hangi sırayla değerlendirileceğini belirler. Sol associativity'de soldaki operatör önce, sağ associativity'de sağdaki operatör önce değerlendirilir. Örnek: $a - b + c$ sol associativity ile $(a - b) + c$ olur.

S2. What is a ternary operator?

Cevap

Ternary operator (üçlü operatör), tam olarak **üç operand** alan bir operatördür. C-tabanlı dillerdeki koşullu operatör ($?:$) bunun klasik örneğidir:

```
koşul ? değer_eğer_doğru : değer_eğer_yanlış
```

Örnek: `average = (count == 0) ? 0 : sum / count;`

S3. What is a prefix operator?

Cevap

Prefix operator, operandından **önce** gelen operatördür. C'deki $++x$ ve $-x$ prefix operatörlere örnektir. Lisp'te tüm operatörler prefix'tir: $(+ a b)$. Perl'de de bazı operatörler prefix'tir.

S4. What operator usually has right associativity?

Cevap

Exponentiation (üstel) operatör ($**$ ya da $^$) genellikle **sağ associativity'e** sahiptir. Örneğin Fortran ve Ruby'de $A ** B ** C$ ifadesi $A ** (B ** C)$ olarak değerlendirilir. C-tabanlı dillerde unary operatörler ($++$, $--$, unary $-$) de sağ associativity'e sahiptir.

S5. What is a nonassociative operator?

Cevap

Nonassociative operator, aynı operatörün ard arda kullanılmasına izin vermeyen operatördür. Bazı dillerde $a < b < c$ yazımına izin verilmez; bu ifade ya derleme hatası verir ya da özel sözdizimi gerektirir. Bu sayede $(a < b) < c$ 'nin matematiksel anlam taşımayan yorumunun önüne geçilir.

S6. What associativity rules are used by APL?**Cevap**

APL'de **tek bir öncelik düzeyi** vardır (tüm operatörler eşit öncelikte) ve **tüm operatörler sağdan sola (right-to-left)** değerlendirilir. Bu nedenle $A * B + C$ ifadesinde önce toplama, ardından çarpma yapılır: $A * (B + C)$ sonucu elde edilir.

S7. What is the difference between the way operators are implemented in C++ and Ruby?**Cevap**

C++: Operatörler derleyici tarafından *fonksiyon çağrısı* olarak implement edilir; özel sözdizimi kullanan birer fonksiyondur.

Ruby: Operatörler tam anlamıyla *method*'tur. Örneğin $a + b$, a 'nın $+$ methodunu b parametresiyle çağırarak eşdeğerdir. Bu sayede operatörler uygulama kodu tarafından doğrudan *override* edilebilir.

S8. Define functional side effect.**Cevap**

Functional side effect (fonksiyonel yan etki), bir fonksiyonun kendi döndüreceği değer dışında **parametrelerinden birini veya global bir değişkeni değiştirmesi** durumudur. Bu durum, expression sonucunun operand değerlendirme sırasına bağımlı hale gelmesine yol açar:

```
1 int a = 5;
2 int fun1() { a = 17; return 3; }
3 // a + fun1() = 8 (a önce okunursa)
4 // a + fun1() = 20 (fun1 önce çalıştırılrsa)
```

S9. What is a coercion?**Cevap**

Coercion (zorunlu dönüşüm), derleyici veya çalışma ortamı tarafından **otomatik olarak yapılan implicit (zımni) tip dönüşümüdür**. Programcı tarafından açıkça belirtilmez. Örneğin Java'da `int` değer `float` beklenen bir yerde kullanılması durumunda derleyici otomatik olarak dönüştürür. Coercion, mixed-mode expression'larda yaygındır ve hata tespitini azaltır.

S10. What is a conditional expression?

Cevap

Conditional expression (koşullu ifade), bir koşulun sonucuna göre iki farklı değer üreten ternary operatörünü kullanan expression'dır. Sözdizimi:

`koşul ? değer1 : değer2`

Örnek: `max = (a > b) ? a : b;` — if-else bloğunu tek satırda ifade eder.

S11. What is an overloaded operator?

Cevap

Overloaded operator (aşırı yüklü operatör), birden fazla anlam için kullanılan operatör sembolüdür. Örneğin `+` operatörü hem tam sayı hem kayan nokta toplamı hem de (Java'da) string birleştirmesi için kullanılır. Kullanıcı tanımlı overloading C++, C# ve F# de programcılara kendi tiplerini operatörlerle donatma imkânı verir.

S12. Define narrowing and widening conversions.

Cevap

Widening conversion: Orijinal tipin tüm değerlerini temsil edebilen daha *büyük* bir tipe dönüşüm. Örnek: `int` → `float`. Genellikle **güvenlidir** ancak bazı durumlarda hassasiyet kaybı olabilir.

Narrowing conversion: Orijinal tipin tüm değerlerini karşılamayan daha *küçük* tipe dönüşüm. Örnek: `double` → `float`. **Veri kaybı veya hata riski** taşır. Java'da sadece widening mixed-mode assignment'a izin verilir.

S13. In JavaScript, what is the difference between `==` and `===`?

Cevap

`==` (loose equality): Operandlar **coerce edilir**; farklı tipler karşılaştırılabilir.

Örnek: `"7" == 7` → `true` (string sayıya çekilir)

`===` (strict equality): Operandlar **coerce edilmez**; tip ve değer aynı olmalıdır.

Örnek: `"7" === 7` → `false` (farklı tipler)

Güvenilir kod için `===` kullanmak önerilir.

S14. What is a mixed-mode expression?

Cevap

Mixed-mode expression, farklı tiplere sahip operandları içeren aritmetik expression'dır. Örneğin bir `int` ile bir `float`'ın aynı expression'da kullanılması. Derleyici, uyumsuz tipleri otomatik coerce eder. Ada ve ML gibi diller bu tür

expression'lara izin vermez; açık tip dönüşümü gereklidir.

S15. What is referential transparency?

Cevap

Referential transparency, bir programda aynı değere sahip iki expression'ın birbirinin yerine **programın çalışmasını değiştirmeden** kullanılabilmesidir. Referentially transparent bir fonksiyonun değeri yalnızca parametrelerine bağlıdır; hiçbir global değişkeni okumaz veya değiştirmez. Pure functional dillerde değişken olmadığı için bu özellik doğal olarak sağlanır.

S16. What are the advantages of referential transparency?

Cevap

- **Anlaşılabilirlik:** Fonksiyonun semantiği bağlamdan (context) bağımsızdır; sadece parametrelerine bakarak ne yaptığını anlamak kolaydır.
- **Test kolaylığı:** Aynı girdiler her zaman aynı çıktıyı üretir.
- **Matematiksel doğruluk:** Fonksiyon matematiksel anlamından farklı davranmaz; doğruluk ispatları daha basittir.
- **Parallelleştirme:** Yan etki olmadığı için fonksiyonlar güvenle paralelleştirilebilir.

S17. How does operand evaluation order interact with functional side effects?

Cevap

Eğer bir fonksiyon çağrısı expression'daki bir değişkeni (parametre veya global) değiştiriyorsa, operandların hangi sırada değerlendirileceği nihai sonucu etkiler. Örneğin `a + fun(a)` ifadesinde `fun`, `a`'yı değiştiriyorsa:

- `a` önce okunursa: eski değer + fonksiyon sonucu
- `fun` önce çağrılırsa: değiştirilmiş `a` + fonksiyon sonucu

Çözümler: side effect'leri yasaklamak (Ada, ML) veya Java gibi soldan sağa sıralama garantisi vermek.

S18. What is short-circuit evaluation?

Cevap

Short-circuit evaluation, bir Boolean expression'ın tüm operand ve operatörleri değerlendirilmeden sonucun belirlenebilmesi durumudur.

- `FALSE && X`: Her zaman `FALSE`; `X` değerlendirilmez.
- `TRUE || X`: Her zaman `TRUE`; `X` değerlendirilmez.

Bu, özellikle null kontrol veya dizi sınır kontrolünde hataları önler.

S19. Name a language that always does short-circuit evaluation. Name one that never does it.

Cevap

Her zaman short-circuit yapan: Java (&&, || operatörleri), Python, Ruby, Perl, ML, F# — bu dillerde tüm logical operatörler short-circuit'dir.

Short-circuit yapmayan (varsayılan olarak): Ada'da `and` ve `or` operatörleri short-circuit **değildir**; `and then` ve `or else` ile açıkça short-circuit talep edilmelidir. C'deki `&` ve `|` bit operatörleri de short-circuit değildir.

S20. How does C support relational and Boolean expressions?

Cevap

C99 öncesinde C'nin ayrı bir **Boolean tipi yoktu**. Bunun yerine sayısal değerler kullanılırdı: `0 = false`, `sıfır olmayan her şey = true`. Relational expression'lar `int` değer üretir (0 veya 1). Herhangi bir sayısal expression Boolean operand olarak kullanılabilirdi; bu durum hata tespitini azaltıyordu. C99 ile birlikte `_Bool` tipi eklendi; `<stdbool.h>` başlığını dahil ederek `bool`, `true`, `false` kullanılabilir hale geldi.

S21. What is the purpose of a compound assignment operator?

Cevap

Compound assignment operator, hedef değişkenin aynı zamanda expression'ın ilk operandı olduğu yaygın atama kalıbını kısaltmak için kullanılır. Örneğin `sum = sum + value` yerine `sum += value` yazılır. Hem **okunurluğu** artırır hem de uzun değişken isimleriyle çalışırken **tekrar yazmayı azaltır**. ALGOL 68 kökenlidir.

S22. What is the associativity of C's unary arithmetic operators?

Cevap

C'deki unary aritmetik operatörler (`++`, `--`, unary `-`, unary `+`) **sağdan sola (right-to-left)** associativity'e sahiptir. Örneğin `-count++` ifadesi önce `count`'u artırır (`count++`), ardından negasyonu uygular: `-(count++)` şeklinde değerlendirilir.

S23. What is one possible disadvantage of treating the assignment operator as if it were an arithmetic operator?

Cevap

En büyük dezavantaj, **atama (=)** ile **eşitlik testi (==)**'in karıştırılmasının derleyici tarafından yakalanamaz hale gelmesidir:

```
1 if (x = y) ... // Hata! Hata! -- y, x'e atanır ve test edilir
2 if (x == y) ... // Doğru
```

Bunun yanı sıra, assignment'ı expression olarak kullanmak kodun okunurluğunu düşürür ve beklenmedik side effect'lere yol açabilir.

S24. What two languages include multiple assignments?**Cevap**

Perl ve **Ruby**, çoklu kaynak ve hedefli atama ifadelerini destekler:

```
1 ($first, $second, $third) = (20, 40, 60); # Perl
2 ($first, $second) = ($second, $first);    # Ruby
   // değişim
```

Python da çoklu hedefli atamayı destekler: `sum = count = 0.`

S25. What mixed-mode assignments are allowed in Java?**Cevap**

Java, yalnızca **widening (genişletme)** coercion gerektiren mixed-mode assignment'lara izin verir:

- `float x = 5;` — `int`'ten `float`'a: **İZİNLI** (widening)
- `int y = 3.5f;` — `float`'tan `int`'e: **YASAK** (narrowing)

Küçük tamsayı literalleri (`byte`, `short`, `char`) için istisna vardır: aralığında olan `int` literalleri bu tiplere atanabilir.

S26. What mixed-mode assignments are allowed in ML?**Cevap**

ML'de mixed-mode assignment **hiçbir şekilde izin verilmez**. Değer bağlamak için kullanılan `val` bildirimi, sol ve sağ tarafın tiplerinin birebir eşleşmesini zorunlu kılar. Farklı tipler arasında dönüşüm yapmak için programcı açıkça **cast (explicit conversion)** kullanmak zorundadır. Bu katı yaklaşım tip güvenliğini ve hata tespitini en üst düzeye çıkarır.

S27. What is a cast?

Cevap

Cast (açık tip dönüşümü), programcı tarafından açıkça istenen explicit tip dönüşümüdür; coercion'ın aksine otomatik değildir:

```
1 (int)angle      // C / C++ / Java / C#  
2 float(sum)     // ML / F#
```

Widening ve narrowing olarak ikiye ayrılır. Narrowing cast'ler veri kaybına yol açabilir ve bazı dillerde uyarı mesajı üretir.

10 Problem Set — Soru Çözümleri

P1. When might you want the compiler to ignore type differences in an expression?

Cevap

- **Hız ve kolaylık:** `int` ile `float`'ı her seferinde elle dönüştürmek zahmetlidir; karma modu otomatik yönetmek kodu kısaltır.
- **Genel amaçlı işlemler:** Bir toplama fonksiyonunun hem tam sayılara hem kayan noktalılara uygulanabilmesi istenebilir.
- **Prototipleme:** Hızlı kod geliştirirken tip tutarlılığına çok dikkat etmek istemeyebilirsiniz.

Ancak bu yaklaşım **hata tespiti azaltır** ve dikkatsiz programlarda beklenmedik tip dönüşümlerine yol açabilir.

P2. State your own arguments for and against allowing mixed-mode arithmetic expressions.

Cevap

Lehine argümanlar:

- Kod daha az verbose olur; her yerde açık cast gerektirmez.
- Matematiksel düşünce biçimimizle uyumludur ($3 + 1.5 = 4.5$).
- Hızlı prototipleme kolaylaşır.

Aleyhine argümanlar:

- Yanlışlıkla yanlış tip kullanılması derleyici hatası vermez; gizli hata oluşur.
- Coercion'ın hangi yönde yapılacağı her zaman sezgisel değildir.
- Güvenlik ve doğruluk açısından açık dönüşüm daha iyi bir yazım alışkanlığı oluşturur.

P3. Do you think the elimination of overloaded operators in your favorite language would be beneficial? Why or why not?

Cevap (Örnek: Java için)

Java'da kullanıcı tanımlı operator overloading yoktur ve bu genellikle **olumlu** görülmektedir:

- Bir operatör gördüğümüzde ne anlama geldiğini bilmek için overload tanımlarını aramak gerekmez — **okunurluk artar**.
- Farklı modüllerin aynı operatörü farklı overload etmesinden kaynaklanan tutarsızlıklar yaşanmaz.

Ancak matris veya karmaşık sayı gibi matematiksel tiplerde `+` ve `*` kullanmak daha doğal ve okunabilir kod sağlardı. Bu nedenle denge önemlidir.

P4. Would it be a good idea to eliminate all operator precedence rules and require parentheses?

Cevap

İyi bir fikir değildir.

Avantajlar:

- Öncelik kurallarını ezberlemek gerekmez; basit ve açık bir sistem.
- APL bu yaklaşımı benimsemiştir.

Dezavantajlar:

- Basit expression'lar bile aşırı parantezli hale gelir.
- **Okunurluk ciddi şekilde düşer**; kod şişerek uzar.
- Matematikte alışmış olduğumuz doğal ifade biçimiyle uyumlu olmaz.

Mevcut öncelik kuralları matematik geleneğinden gelir ve kodun daha doğal okunmasını sağlar.

P5. Should C's assigning operations (e.g., $+=$) be included in other languages?

Cevap

Evet, alınmalıdır. Nedenleri:

- Yaygın bir kalıp olan $x = x + y$ yerine $x += y$ daha kısadır.
- Uzun değişken isimlerinde tekrar yazmaktan kaçınılır.
- Okunurluğu artırır ve yazım hatalarını azaltır.
- Pek çok modern dil (Python, JavaScript, Ruby) zaten benimsemiştir.

P6. Should C's single-operand assignment forms (e.g., $++count$) be included in other languages?

Cevap

Tartışmalıdır.

Lehine: Döngü sayaçlarında $i++$ çok yaygınlaşmıştır ve kısadır.

Aleyhine: Prefix ($++count$) ve postfix ($count++$) arasındaki ince anlam farkı sürpriz hatalara yol açabilir. Python kasıtlı olarak bu operatörleri **dahil etmemiş** ve $count + = 1$ yazımını tercih etmiştir; bu daha açık ve güvenlidir.

P7. Describe a situation in which the add operator would not be commutative.

Cevap

Java'da $+$ operatörü string birleştirme için de kullanılır ve bu durumda **commutative** (değişimli) değildir:

```

1 "Hello" + " World" // "Hello World"
2 " World" + "Hello" // " WorldHello" -- ıFarklı sonuç!

```

Ayrıca kullanıcı tanımlı operator overloading'de programcı '+'ı commutative olmayan bir şekilde tanımlayabilir.

P8. Describe a situation in which the add operator would not be associative.

Cevap

Kayan nokta (floating-point) sayılarda aritmetik işlemler her zaman tam olarak temsil edilemez. Çok büyük/küçük sayılarla çalışırken:

```

1 // A ve C çok büyük pozitif, B ve D çok büyük negatif
2 (A + B) + (C + D) // şTama olmayabilir
3 (A + C) + (B + D) // şTama olabilir!

```

Java'da string birleştirme de associative değildir:

```

1 (1 + 2) + "3" // "33"
2 1 + (2 + "3") // "123"

```

P9. Show the order of evaluation with the given precedence/associativity rules.

Verilen kurallar (soldan sağa associativity):

Öncelik	Operatörler
En Yüksek	*, /, not
2	+, -, &, mod - (unary)
4	=, / =, <, < =, > =, > and, or, xor

(a) $a * b - 1 + c$

1. $a * b$ (en yüksek öncelik: *)
2. $(a * b) - 1$ (soldan sağa: -)
3. $((a * b) - 1) + c$ (soldan sağa: +)

Parantezli gösterim: $((a * b)^1 - 1)^2 + c)^3$

(b) $a * (b - 1) / c \bmod d$

1. $(b - 1)$ (parantez önce)
2. $a * (b - 1)$ (soldan sağa)
3. $(a * (b - 1)) / c$ (soldan sağa)
4. $((a * (b - 1)) / c) \bmod d$ (soldan sağa)

Gösterim: $((a * (b - 1)^1) / c)^3 \bmod d)^4$

(c) $(a - b) / c \ \& \ (d * e / a - 3)$

1. $(a - b)$ (parantez)
2. $d * e$ (yüksek öncelik: $*$)
3. $(d * e) / a$ (soldan sağa: $/$)
4. $((d * e) / a) - 3$ (toplama/çıkarma düzeyi)
5. $(a - b) / c$ (bölme)
6. $((a - b) / c) \ \& \ ((d * e / a) - 3)$ ($\&$ en son)

(d) $-a \text{ or } c = d \text{ and } e$

Öncelik: unary $-$ $>$ relational $=$ $>$ and $>$ or

1. $(-a)$ (unary minus — en yüksek)
2. $c = d$ (relational)
3. $(c = d) \text{ and } e$ (and, or'dan yüksek)
4. $(-a) \text{ or } ((c = d) \text{ and } e)$ (or — en düşük)

(e) $a > b \text{ xor } c \text{ or } d < = 17$

Öncelik: relational $>$ xor $/$ or

1. $a > b$ (relational)
2. $d < = 17$ (relational)
3. $(a > b) \text{ xor } c$ (xor, or'dan önce — soldan sağa)
4. $((a > b) \text{ xor } c) \text{ or } (d < = 17)$ (or — en düşük)

(f) $-a + b$

Unary $-$ $>$ $+$

1. $(-a)$ (unary minus)
2. $(-a) + b$ (binary toplama)

Gösterim: $((-a)^1 + b)^2$

P10. Show the order assuming no precedence, right-to-left associativity.

Cevap

Öncelik yoksa ve tüm operatörler sağdan sola değerlendirilirse:

(a) $a * b - 1 + c$: En sağdaki önce: $a * (b - (1 + c)^1)^2)^3$

(b) $a * (b - 1) / c \text{ mod } d$: Parantez önce; kalan sağdan sola: $a * ((b - 1) /$

$$(c \text{ mod } d)^1)^2)^3$$

(c)–(f): Benzer şekilde her zaman sağdaki operatör önce değerlendirilir. Parantezli kısımlar her durumda önce tamamlanır.

P11. Write a BNF description of the precedence and associativity rules.

Cevap — BNF (sadece a, b, c, d, e operandları)

```

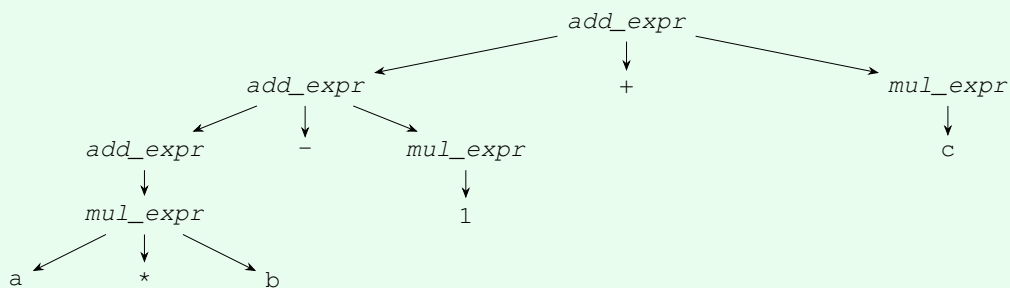
1 <expr>      -> <expr> or <and_expr>
2              | <expr> xor <and_expr>
3              | <and_expr>
4 <and_expr>   -> <and_expr> and <rel_expr>
5              | <rel_expr>
6 <rel_expr>   -> <add_expr> <relop> <add_expr>
7              | <add_expr>
8 <relop>      -> = | / = | < | < = | > = | >
9 <add_expr>   -> <add_expr> + <mul_expr>
10             | <add_expr> - <mul_expr>
11             | <add_expr> & <mul_expr>
12             | <add_expr> mod <mul_expr>
13             | <mul_expr>
14 <mul_expr>   -> <mul_expr> * <unary_expr>
15             | <mul_expr> / <unary_expr>
16             | <mul_expr> not <unary_expr>
17             | <unary_expr>
18 <unary_expr>-> - <primary>
19             | <primary>
20 <primary>   -> a | b | c | d | e

```

Sol özyinelemeli kurallar sol associativity'yi sağlar. Her katman bir öncelik düzeyine karşılık gelir.

P12. Draw parse trees for the expressions of Problem 9.

Cevap — (a) $a * b - 1 + c$ için Parse Ağacı



Diğer expression'lar için benzer ağaçlar, BNF kurallarına göre sol özyinelemeli şekilde inşa edilir.

P13. What are the values of sum1 and sum2?

Cevap

```

1 int fun(int* k) { *k += 4; return 3 * (*k) - 1; }
2 // i = 10, j = 10
3 sum1 = (i / 2) + fun(&i);
4 sum2 = fun(&j) + (j / 2);

```

(a) Soldan sağa:

- **sum1:** Önce $i/2 = 5$, sonra $\text{fun}(\&i)$: $i = 14$, döner $3 \cdot 14 - 1 = 41 \Rightarrow \text{sum1} = 5 + 41 = 46$
- **sum2:** Önce $\text{fun}(\&j)$: $j = 14$, döner 41, sonra $j/2 = 7 \Rightarrow \text{sum2} = 41 + 7 = 48$

(b) Sağdan sola:

- **sum1:** Önce $\text{fun}(\&i)$: $i = 14$, döner 41, sonra $i/2 = 7 \Rightarrow \text{sum1} = 7 + 41 = 48$
- **sum2:** Önce $j/2 = 5$, sonra $\text{fun}(\&j)$: $j = 14$, döner 41 $\Rightarrow \text{sum2} = 41 + 5 = 46$

P14. What is your primary argument against the operator precedence rules of APL?

Cevap

Aleyhine ana argüman: APL'nin tek düzey öncelik + sağdan sola associativity tasarımı **matematiksel sezgiye aykırıdır**. $A * B + C$ ifadesi $A * (B+C)$ olarak değerlendirilir; oysa matematik geleneği $(A*B) + C$ der. Bu durum, diğer dillerden gelen programcıların APL kodunu okurken sürekli zihinsel ek çaba harcamasına neden olur ve **hata oluşturma riskini artırır**.

P15. Explain why it is difficult to eliminate functional side effects in C.

Cevap

C, yalnızca fonksiyon yapısı içeren bir dildir ve fonksiyonlar **tek bir değer döndürür**. Birden fazla değer iletmek için:

- **Pointer parametreler** kullanılması gerekir (bu side effect'tir).
- **Global değişkenlere erişim** çoğunlukla verimlilik için zorunludur (örneğin derleyicilerdeki sembol tablosu).
- Struct ile birden fazla değer döndürmek mümkün olsa da bu yaklaşım kodun okunabilirliğini ve performansını olumsuz etkiler.

Bu nedenle C'de side effect'leri tamamen yasaklamak, dilin temel kullanım kalıplarını işlevsiz kılar.

P16. Make up a list of operator symbols to eliminate all operator overloading.

Cevap (Örnek: Java için)

Operasyon	Mevcut	Ayrı Sembol
Tam sayı toplama	+	+i
Float toplama	+	+f
String birleştirme	+	^^
Tam sayı çarpma	*	*i
Float çarpma	*	*f
Tam sayı bölme	/	/i
Float bölme	/	/f

Bu yaklaşım **okunurluğu ciddi şekilde düşürür**; bu nedenle pratikte benimsenmez.

P17. Determine whether narrowing explicit type conversions provide error messages.

Cevap

Java: Narrowing cast için derleme zamanı uyarısı **vermez**; programcı açıkça cast yazmak zorundadır. Çalışma zamanında da hata mesajı yoktur:

```
1 double d = 1.3e25;
2 int i = (int) d; // Derlenir, ışıçalır; i anlamsız bir değer
   alır
```

Python: Implicit narrowing yoktur; tüm dönüşümler açık fonksiyon çağrılarıyla yapılır ve genellikle hata mesajı üretmez, yalnızca değeri keser:

```
1 int(3.7) # 3 döner, sessizce keser
```

P18. Should an optimizing compiler for C or C++ be allowed to change the order of subexpressions in a Boolean expression?

Cevap

Hayır, olmamalıdır. Neden:

- C/C++'ta `&&` ve `||` **short-circuit** semantiği garanti eder. Derleyici sıralamayı değiştirirse bu garantiyi bozar.
- Bir operandın side effect'i varsa (örneğin `++` veya fonksiyon çağrısı), sıralama sonucu değiştirebilir.
- Bazı kontroller sıralama bağımlıdır: `p != NULL && p->val > 0` — sıralama değişirse null pointer hatası!

P19. What is the value of x after the assignment in main?

Cevap

```
1 int fun(int *i) { *i += 5; return 4; }
2 void main() { int x = 3; x = x + fun(&x); }
```

(a) Soldan sağa:

Önce `x` okunur: `x = 3` Sonra `fun (&x) : x = 8`, döner 4.
`x = 3 + 4 = 7`

(b) Sağdan sola:

Önce `fun (&x) : x = 8` döner 4. Sonra `x` okunur: `x = 8`.
`x = 8 + 4 = 12`

P20. Why does Java specify left-to-right operand evaluation order?

Cevap

Java, **platform bağımsızlığı ve deterministik davranış** sağlamak için operand değerlendirme sırasını soldan sağa olarak garanti eder. Bu sayede:

- Aynı kod farklı JVM implementasyonlarında aynı sonucu üretir.
- Side effect içeren expression'lar tahmin edilebilir davranır.
- Programcının platformlar arası fark yüzünden hata ayıklaması gerekmez.

Bu, optimizasyon olanaklarının bir kısmından fedakârlık anlamına gelir; ancak **güvenilirlik ve taşınabilirlik** ön plana çıkarılmıştır.

P21. Explain how the coercion rules of a language affect its error detection.

Cevap

Kapsamlı coercion kuralları olan dillerde (C, C++) derleyici, uyumsuz tip kombinasyonlarını otomatik olarak dönüştürür. Bu durum:

- **Gerçek yazım hatalarını gizler:** Örneğin `d = b * a` yerine `c` yazılması gerekirken; `int a` otomatik coerce edildiği için derleyici hata vermez.
- **Tip kontrol etkinliğini azaltır:** Tip sisteminin amacı olan hata tespiti zayıflar.

Ada ve ML gibi dillerde coercion çok kısıtlanmış veya yoktur; tip uyumsuzluğu **derleme zamanı hatası** olarak yakalanır ve güvenilirlik artar.

Chapter 7 — Konu Anlatım Föyü Tamamlandı